

دليل محترفي جافا: المراجع الشامل للنصوص

الجزء 4: دالات ال String الأساسية | الجزء 5: قوة ال StringBuilder وكفاءة الذاكرة

4. Common String Methods (دوال النصوص الشائعة)

النصوص في جافا تتميز بأنها **immutable** (غير قابلة للتغيير في الذاكرة). عندما تستدعي أي دالة تعديل على ال String، فإن جافا لا تغير النص الأصلي بل تقوم ببناء كائن (Object) جديد تماماً في الذاكرة وإرجاعه. إليك الشرح المعمق لأهم 15 دالة يجب على كل مطور إتقانها:

length()

Return: int

الشرح العميق: تحسب عدد ال Unicode code units (الحروف والرموز والفراغات) المكونة للنص. الدالة تعتمد على مصفوفة الحروف الداخلية وتعمل في زمن ثابت **O(1)**.

الاستخدام: فحص قيود المدخلات (مثل: ألا يقل طول كلمة المرور عن 8 خانات).

```
String pass = "Admin@123";  
System.out.println(pass.length()); // المخرج: 9
```

charAt(int index)

Return: char

الشرح العميق: الحرف المقابل للفهرس (Index) الممرر، والذي يـ **0** مؤينته عند **length() - 1**. تمرير كود غير موجود يرمي **StringIndexOutOfBoundsException**.

الاستخدام: معالجة الحروف بشكل منفرد أو فحص صيغة الحرف الأول.

```
String word = "Java";  
System.out.println(word.charAt(0)); // المخرج: J
```

substring(int beginIndex, int endIndex)

Return: String

الشرح العميق: تقطع جزءاً من النص. الفهرس `beginIndex` مشمول (Inclusive)، بينما الفهرس `endIndex` غير مشمول. إذا لم تمرر المعامل الثاني، ستقطع الدالة حتى نهاية النص. (Exclusive)

الاستخدام: فصل البيانات المركبة مثل استخراج اسم المستخدم من البريد الإلكتروني.

```
String email = "dev@java.com";
System.out.println(email.substring(0, 3)); // المخرج: dev
System.out.println(email.substring(4));    // المخرج: java.com
```

toUpperCase() | toLowerCase()

Return: String

الشرح العميق: تحويل النص بالكامل إلى أحرف كبيرة أو صغيرة. لا تؤثر الدالتان على الحروف العربية أو الأرقام والرموز.

الاستخدام: توحيد النصوص قبل مقارنتها لجعل المقارنة غير حساسة لحالة الأحرف (Case-insensitive).

```
String role = "Admin";
System.out.println(role.toLowerCase()); // المخرج: admin
```

trim() | strip()

Return: String

الشرح العميق: إزالة المسافات من البداية والنهاية. `trim()` تزيل فقط الحروف التي لها كود ASCII أقل من أو يساوي 32 (الفراغ التقليدي). `strip()` (أضيفت في Java 11) فهي متوافقة مع معيار Unicode وتتعرف على كافة أنواع الفراغات بمختلف اللغات وتزيلها.

```
String input = "  جافا  ";
System.out.println(input.trim().length()); // قد لا يحذف فراغ اليونيكود
System.out.println(input.strip().length()); // يحذف كل شيء ويعيد الطول الصافي
```

contains(CharSequence s)

Return: boolean

الشرح العميق: تبحث عما إذا كان النص يحتوي على تسلسل معين من الحروف. حساسة لحالة الأحرف.

```
String phrase = "Learn Java step by step";  
System.out.println(phrase.contains("Java")); // true
```

startsWith(String prefix) | endsWith(String suffix)

Return: boolean

الاستخدام: فحص أمان الروابط (تبدأ بـ `https://`) أو التحقق من نوع وامتداد الملفات المقبولة (تنتهي بـ `.pdf`).

```
String file = "report.pdf";  
System.out.println(file.endsWith(".pdf")); // true
```

indexOf(String str)

Return: int

الشرح العميق: تبحث عن أول ظهور للنص الفرعي الممرر وتعيد رقم الفهرس الخاص به. ****إذا لم تجده مطلقاً، فإنها تُرجع القيمة -1.**

```
String info = "id:101,name:ali";  
System.out.println(info.indexOf("name:")); // المخرج: 7  
System.out.println(info.indexOf("age")); // المخرج: -1
```

replace(CharSequence target, CharSequence replacement)

Return: String

الشرح العميق: يقوم باستبدال جميع التطابقات للنص القديم بالنص الجديد داخل السلسلة وتعيد السلسلة الجديدة المعدلة.

```
String text = "I love Java. Java is cool.";
System.out.println(text.replace("Java", "Kotlin")); // المخرج: I love Kotlin.
```

split(String regex)

Return: String[]

الشرح العميق: ينقسم النص إلى مصفوفة نصوص بناءً على تعبير نمطي (Regex) يمثل الفاصل المطلوب.

```
String csv = "Apple,Banana,Orange";
String[] fruits = csv.split(","); // تفصلهم عند كل فاصلة
```

isEmpty() | isBlank()

Return: boolean

الشرح العميق: `isEmpty()` تعيد true فقط إذا كان الطول يساوي صفر `""`. بينما `isBlank()` (منذ Java 11) تعيد true إذا كان النص فارغاً تماماً `**` أو يحتوي على مسافات وفراغات فقط `" "`.

تنبيه قاتل: تأكد دائماً أن الكائن ليس `null` قبل استدعاء هاتين الدالتين تجنباً لـ `NullPointerException`.

5. **StringBuilder** (المرن النصّ)

على عكس كلاس String الثابت، فإن كلاس **StringBuilder** يُعتبر **Mutable** (قابل للتعديل والتغيير مباشرة في الذاكرة). يقوم داخلياً بإدارة مصفوفة حروف ديناميكية قابلة للتوسع دون الحاجة لإنشاء كائنات جديدة في كل عملية تعديل، مما يجعله الخيار الأول لتحسين الأداء وكفاءة الذاكرة.

لماذا ومتى نستخدم **StringBuilder**؟ (المشكلة والحل)

عند دمج النصوص باستخدام معامل الزائد (+) داخل حلقة تكرارية (Loop)، تقوم جافا بإنشاء كائن String جديد كلياً في كل دورة للمرور، مما يستهلك الذاكرة (Heap Memory) ويبطئ البرنامج بشكل ملحوظ بسبب استدعاء الـ Garbage Collector باستمرار لتنظيف الذاكرة المهترئة.

⚠️ الطريقة الخاطئة والكارثية (باستخدام String + داخل Loop):

```
String result = "";
for (int i = 0; i < 10000; i++) {
    result += i; // إنشاء 10000 كائن
}
```

✅ الطريقة الاحترافية والصحيحة (باستخدام **StringBuilder**):

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) {
    sb.append(i); // سرعة هائلة
}
String finalResult = sb.toString();
```

أهم دالات كلاس StringBuilder وشرحها المعمق:

append(X x) Return: StringBuilder

الشرح: تقوم بإضافة القيمة الممررة (سواء كانت نصاً، رقماً، حرفاً، أو بولين) إلى نهاية النص الحالي مباشرة داخل البافر (Buffer).

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World").append(2026); // الـ Chaining دعم
System.out.println(sb); // المخرج: Hello World2026
```

insert(int offset, X x) Return: StringBuilder

الشرح: تقوم بحشر أو إدخال القيمة الممررة عند فهرس (Index) محدد، وتقوم بإزاحة الحروف المتبقية تلقائياً إلى اليمين.

```
StringBuilder sb = new StringBuilder("Javat");
sb.insert(4, "Scrip");
System.out.println(sb); // المخرج: JavaScript
```

delete(int start, int end) Return: StringBuilder

الشرح: تقوم بحذف الحروف ابتداءً من الفهرس **start** (مشمول) وحتى الفهرس **end** (غير مشمول).

```
StringBuilder sb = new StringBuilder("Java programming");
sb.delete(4, 16);
System.out.println(sb); // المخرج: Java
```

reverse()

Return: `StringBuilder`

الشرح: تقوم بعكس ترتيب الحروف بالكامل داخل النص في مكانها (In-place)، وهي دالة خوارزمية سريعة جداً ومفيدة في حل التحديات البرمجية.

```
StringBuilder sb = new StringBuilder("عربي");
sb.reverse();
System.out.println(sb); // المخرج: بيرع
```

toString()

Return: `String`

الشرح: دالة التحويل النهائية. بعد الانتهاء من بناء وتعديل النص بكفاءة باستخدام `StringBuilder`، نستدعي هذه الدالة لتحويل محتوى البافر إلى كائن `String` تقليدي غير قابل للتعديل لكي تتمكن من تمريره للدوال الأخرى أو حفظه.

```
String finalResult = sb.toString();
```

مقارنة ملخصة (String مقابل StringBuilder)

الميزة	كلاس String	كلاس StringBuilder
القابلية للتعديل (Mutability)	(غير قابل للتعديل) Immutable	(قابل للتعديل في مكانه) Mutable
استهلاك الذاكرة في الحلقات (Loops)	عالٍ جداً وضار بالأداء (ينشئ كائنات مكررة)	منخفض ومثالي (يعدل على نفس الكائن)
الأداء والسرعة	أبطأ عند التعديل والدمج المستمر	سريع جداً وخفيف